BÖLÜM

2

Toolbox ve Genel Kontroller



BU BÖLÜMÜN SONUNDA

- Kontrol, sınıf ve nesne kavramları arasındaki ilişkiyi açıklayabilecek,
- Properties penceresinden ve kod ile kontrol özelliklerini değiştirebilecek,
- Visible ile Enabled arasındaki farkı ayırt edebilecek,
- Toolbox'taki genel kontrollerin amacını, temel özelliklerini ve olaylarını bilecek,
- Bir ihtiyaç için en uygun kontrolü seçip gerekçelendirebilecek,
- Kod ile çalışma anında kontrol oluşturabileceksiniz.

KampüsCepte · öğrenciler için ücretsiz ders notları

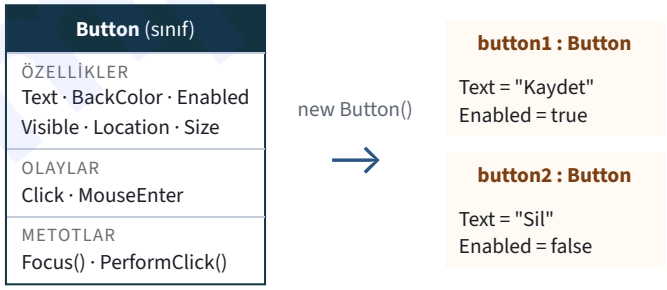
Bir form uygulamasının yüzü kontrollerdir. Butonlar, metin kutuları, listeler, takvimler... Bu bölümde Toolbox'taki genel kontrolleri tanıyor, her birinin hangi ihtiyaca cevap verdiğini ve en çok kullanılan özelliklerini öğreniyoruz.

2.1 Kontrol, Sınıf ve Nesne

Toolbox'ın **Common Controls** bölümünde forma eklenebilecek genel kontroller listelenir. C#'taki her kontrol, `System.Windows.Forms` içindeki **Control** sınıfından türetilmiş bir sınıftır. Toolbox'tan bir Button sürükleyip forma bıraktığımızda Visual Studio, `Button` sınıfından `button1` adında bir **nesne** oluşturur. İkinci butonu bıraktığımızda aynı sınıftan ikinci bir nesne (`button2`) oluşur. Sınıf tektir, nesneler istendiği kadar çoğaltılabilir.



Tek kalıp (sınıf), birbirinden farklı süslenmiş ürünler (nesneler)



Şekil 2.1 *Button sınıfı ve bu sınıftan üretilmiş iki nesne. Her nesnenin özellik değerleri kendine aittir.*

Kavram	Anlamı	Örnek
Özellik (property)	Nesnenin bir niteliği; okunabilir ve çoğu zaman değiştirilebilir.	`button1.Text = "Kaydet";`
Olay (event)	Nesnenin başına gelen ve koda haber verilen durum.	`button1_Click`
Metot (method)	Nesneye yaptırılan iş; adının sonunda parantez bulunur.	`textBox1.Clear();`

KampüsCepte · öğrenciler için ücretsiz ders notları

NOT

Kontrolün kodda kullanılan adı **(Name)** özelliğidir (`button1` , `textBox1` ...). Ekranda görünen yazı ise **Text** özelliğidir. Büyük projelerde adlar anlamlı verilir: `btnKaydet` , `txtAd` , `lblSonuc` gibi. Bu kitapta, ders sunumlarıyla uyumlu olması için Visual Studio'nun varsayılan adları kullanılmıştır.

2.2 Properties Penceresi ve Ortak Özellikler

Bir kontrolün özellikleri iki yoldan değiştirilebilir. **Tasarım anında** kontrol seçilir ve sağ alttaki **Properties** penceresinden istenen özellik ayarlanır. **Çalışma anında** ise aynı özellik kod ile değiştirilir. Properties penceresinde yapılan her ayar, Visual Studio tarafından `Form1.Designer.cs` dosyasına kod olarak yazılır. Yani iki yol aslında aynı işi yapar.

Özellik	Görevi	Kod ile örnek
`Text`	Kontrolün üzerinde/içinde görünen yazı	`button1.Text = "Tamam";`
`BackColor`	Arka plan rengi	`this.BackColor = Color.Red;`
`ForeColor`	Yazı (karakter) rengi	`label1.ForeColor = Color.Blue;`
`Visible`	Kontrolün görünür (`true`) ya da görünmez (`false`) olması	`button3.Visible = false;`
`Enabled`	Kontrolün aktif (`true`) ya da pasif (`false`) olması	`button4.Enabled = false;`
`Font`	Yazı tipi ve boyutu	`new Font("Arial", 12)`
`Location`	Kontrolün form üzerindeki konumu (x; y)	`new Point(20, 40)`
`Size`	Genişlik ve yükseklik	`new Size(120, 30)`
`RightToLeft`	Yazının sağdan sola yazılması	`RightToLeft.Yes`
`TabIndex`	Tab tuşuyla kontroller arasında geçiş sırası	`textBox2.TabIndex = 1;`
`ContextMenuStrip`	Sağ tıklayınca açılacak menüyü kontrole bağlar	Tasarımda seçilir

DİKKAT

Kontrolü aktif/pasif yapan özelliğin adı `Enable` değil `Enabled` 'dır. Properties penceresinde de *Enabled* olarak görünür. `button1.Enable = false;` yazılırsa "Button sınıfında Enable adında bir tanım yok" hatası alınır.

2.3 Text, BackColor, Visible ve Enabled Uygulaması

1. adım: Forma 4 buton eklenir ve Properties penceresinin **Text** kısmından adları sırasıyla TEXT, BACKCOLOR, VISIBLE ve ENABLE yapılır. **2. adım:** Her butona çift tıklanarak aşağıdaki kodlar yazılır ve program F5 ile çalıştırılır.

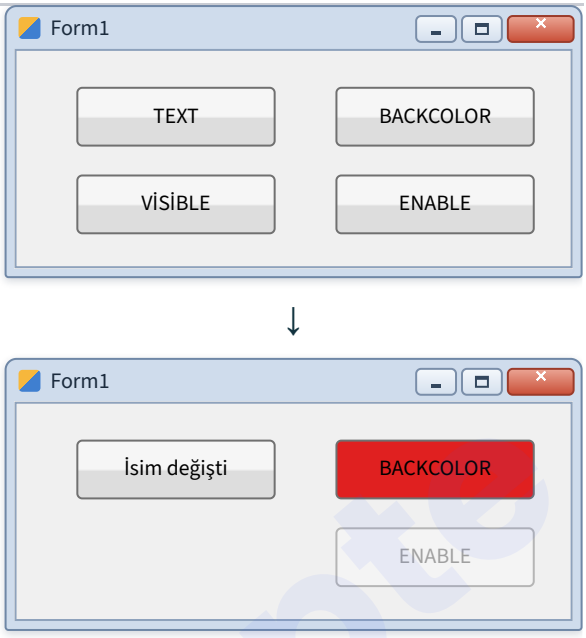
KampüsCepte · öğrenciler için ücretsiz ders notları

```
private void button1_Click(object sender,
EventArgs e)
{
    button1.Text = "İsim değişti";
}

private void button2_Click(object sender,
EventArgs e)
{
    button2.BackColor = Color.Red;
}

private void button3_Click(object sender,
EventArgs e)
{
    button3.Visible = false;   // buton tamamen
görünmez olur
}

private void button4_Click(object sender,
EventArgs e)
{
    button4.Enabled = false;   // buton görünür
kalır ama tıklanamaz
}
```



Dört butona da tıklandıktan sonra (alttaki görüntü): birinci butonun yazısı değişmiş, ikincisi kırmızı olmuş, üçüncüsü tamamen **kaybolmuş**, dördüncüsü ise yerinde duruyor ama **soluk** görünüyor ve artık tıklanamıyor.

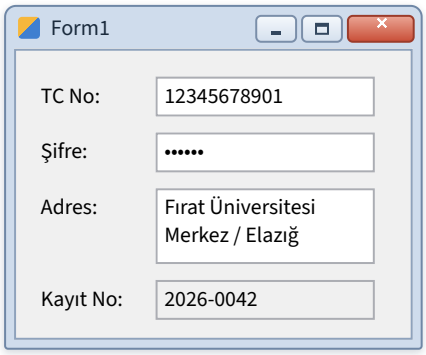
	Visible = false	Enabled = false
Ekranda	Görünmez, yeri boş kalır	Görünür, soluk (gri) çizilir
Tıklanabilir mi?	Hayır (görülmüyor ki)	Hayır, olayları tetiklenmez
Kullanım örneği	Yetkisiz kullanıcıdan bir butonu tamamen gizlemek	Form doldurulmadan Kaydet'e basılmasını engellemek

3. adım: Aynı işlemler kod yazmadan, her butonun Properties penceresindeki **Text**, **BackColor**, **Visible** ve **Enabled** satırlarından da yapılabilir. Fark şudur: Properties penceresinden yapılan ayar formun **açılış** durumunu belirler, kod ise durumu program **çalışırken** değiştirir.

KampüsCepte · öğrenciler için ücretsiz ders notları

TextBox'a özgü bazı özellikler

Özellik	Görevi
`MaxLength`	Yazılabilecek en fazla karakter sayısı (varsayılan 32767)
`Multiline`	`true` ise kutu birden çok satır alır
`PasswordChar`	Yazılan karakterleri gizler, ör. `'*'`
`ReadOnly`	`true` ise kutu yalnızca okunur, içine yazılamaz



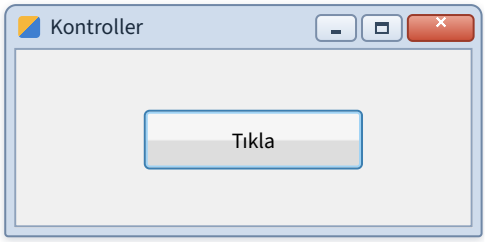

```
private void Form1_Load(object sender, EventArgs e)
{
    textBox1.MaxLength = 11;          // en fazla 11 karakter (ör. TC kimlik no)
    textBox2.PasswordChar = '*';      // yazılanlar * olarak görünür
    textBox3.Multiline = true;        // birden çok satır yazılabilir
    textBox4.ReadOnly = true;         // yalnızca okunur, kullanıcı değiştiremez
}
```

2.4 Genel Kontroller

Bu bölümde Toolbox'taki genel kontroller tek tek ele alınmıştır. Her kontrol için kısa bir tanım, en çok kullanılan özellikler ve form üzerindeki görünüm verilmiştir. Kod örneği gerektirenlerde örnek kod da eklenmiştir. Kontroller Toolbox'taki alfabetik sırayla verilmiştir.

1. Button

Genellikle en çok kullanılan tetikleyici kontroldür. Üzerine çift tıklandığında varsayılan olarak açılan olay **Click**'tir.



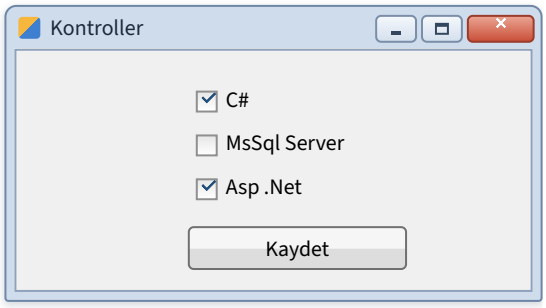
Özellik / Üye	Açıklama
`Text`	Buton yazısı
`TextAlign`	Yazının buton içindeki hizası (ör. `ContentAlignment.TopRight`)
`FlatStyle`	Görünüm biçimi: Flat, Popup, Standard, System

> **NOT**
>
> Sunumlarda özelliğin adı *FlatSytle* diye geçmektedir. Doğru yazım **FlatStyle**'dır.

KampüsCepte · öğrenciler için ücretsiz ders notları

2. CheckBox

Kullanıcıya metin içeren bir seçenek sunar (evet/hayır, doğru/yanlış gibi). Birden fazla CheckBox aynı anda işaretlenebilir. İşaret durumu değişince **CheckedChanged** olayı tetiklenir.

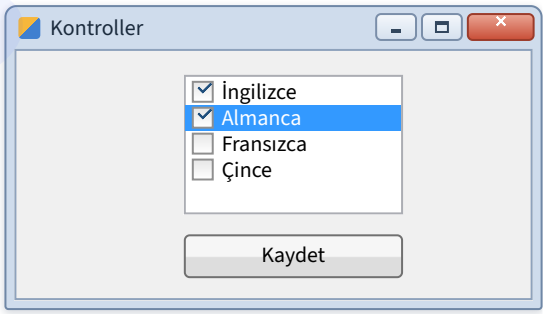


Özellik / Üye	Açıklama
`Text`	Kutunun yanındaki yazı
`Checked`	İşaretli ise `true`
`CheckAlign`	Kutucuğun yazıya göre konumu

```
private void button1_Click(object sender, EventArgs e)
{
    string secilenler = "";
    if (checkBox1.Checked) secilenler += checkBox1.Text + "  ";
    if (checkBox2.Checked) secilenler += checkBox2.Text + "  ";
    if (checkBox3.Checked) secilenler += checkBox3.Text + "  ";
    MessageBox.Show("Seçilenler: " + secilenler);
}
```

3. CheckedListBox

CheckBox'a benzer, ancak tek bir kontrolde birden fazla işaret kutusu oluşturmayı sağlar. Elemanlarına bir dizinin elemanlarıymış gibi indisle erişilir.



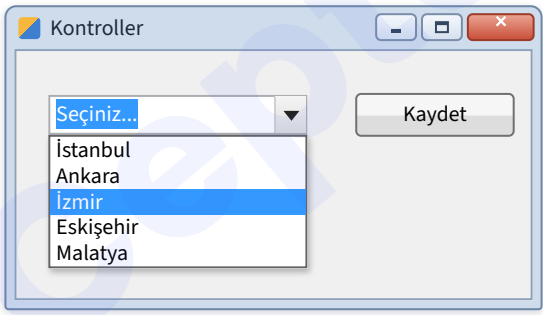
Özellik / Üye	Açıklama
`Items`	Eleman listesi
`SelectedItem`	/ `SelectedIndex` seçili (mavi) eleman ve indisi
`CheckedItems`	İşaretli elemanların koleksiyonu
`GetItemChecked(i)`	**Metottur.** i. eleman işaretliyse `true` döndürür
`SetItemChecked(i, d)`	i. elemanın işaretini değiştirir

KampüsCepte · öğrenciler için ücretsiz ders notları

```
private void button1_Click(object sender, EventArgs e)
{
    string diller = "";
    foreach (object dil in checkedListBox1.CheckedItems)
    {
        diller += dil + "\n";
    }
    MessageBox.Show("İşaretlenen diller:\n" + diller);

    // Tek bir elemanın durumunu sorgulama ve değiştirme (indis 0'dan başlar):
    bool almancaSecili = checkedListBox1.GetItemChecked(1);
    checkedListBox1.SetItemChecked(0, true);
}
```

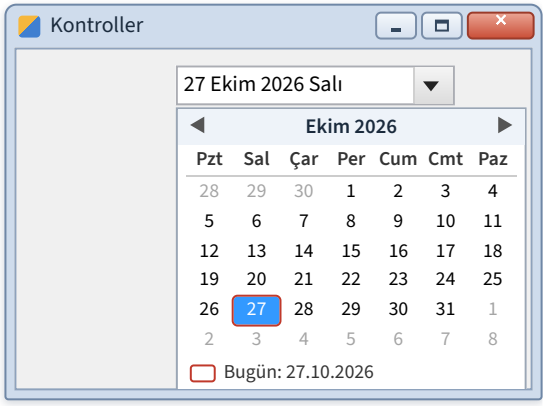
4. ComboBox



Aşağı doğru açılan bir seçim listesi sunar. Listedeki elemanlar bir dizinin elemanları gibi indislidir. ComboBox ile yapılan ekleme ve silme işlemleri 4. bölümde ayrıntılı işlenecektir.

Özellik / Üye	Açıklama
`Text`	Kutuda görünen metin (ör. *Seçiniz...*)
`Items`	Elemanlar: `Add`, `AddRange`, `Clear`, `Remove`
`SelectedItem`	Seçili eleman
`SelectedIndex`	Seçili elemanın indisi; seçim yoksa `-1`
`DropDownStyle`	`DropDownList` yapılırsa kutuya elle yazılamaz
`MaxDropDownItems`	Açılan listede aynı anda görünen eleman sayısı

5. DateTimePicker



Açılan kutu içinde bir takvim görünümü sunar. Bugünün tarihini gösterir ve istenen herhangi bir tarihin seçilmesini sağlar. Tarih değişince **ValueChanged** olayı tetiklenir.

Özellik / Üye	Açıklama
`Value`	Seçili tarih (`DateTime` türünde)
`MinDate`	/ `MaxDate` seçilebilecek en erken / en geç tarih
`Format`	Long, Short, Time ya da Custom
`CustomFormat`	Format Custom iken kalıp, ör. `"dd.MM.yyyy"`

KampüsCepte · öğrenciler için ücretsiz ders notları

DİKKAT

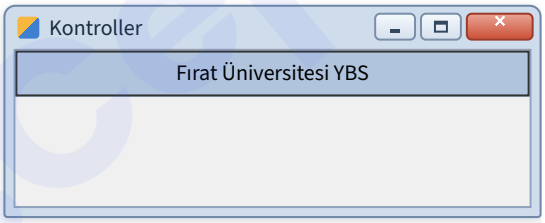
Bazı kaynaklarda DateTimePicker'ın özellikleri arasında *MaxDropDownItems* sayılır. Bu özellik DateTimePicker'da **yoktur**, ComboBox'a aittir. Kod içinde kullanılırsa derleme hatası verir.

```
private void Form1_Load(object sender, EventArgs e)
{
    dateTimePicker1.MaxDate = DateTime.Today;          // ileri bir tarih seçilemez
    dateTimePicker1.Format = DateTimePickerFormat.Custom;
    dateTimePicker1.CustomFormat = "dd.MM.yyyy";
}

private void button1_Click(object sender, EventArgs e)
{
    DateTime secilen = dateTimePicker1.Value;
    MessageBox.Show("Seçilen tarih: " + secilen.ToShortDateString());
}
```

6. Label

Form üzerinde kullanıcıya bilgi vermek, kutuların ne işe yaradığını yazmak ya da bir sonucu göstermek için kullanılır. Kullanıcı Label'ın içine yazı yazamaz.

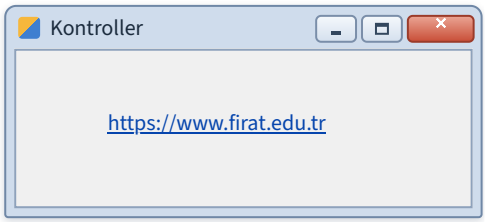


Özellik / Üye	Açıklama
`Text`	Görünen yazı
`BorderStyle`	Kenarlık: None, FixedSingle, Fixed3D
`BackColor`	Arka plan rengi
`AutoSize`	`true` ise boyut yazıya göre ayarlanır

```
private void Form1_Load(object sender, EventArgs e)
{
    label1.Text = "Fırat Üniversitesi YBS";
    label1.AutoSize = false;
    label1.BorderStyle = BorderStyle.FixedSingle;
    label1.BackColor = Color.LightSteelBlue;
}
```


KampüsCepte · öğrenciler için ücretsiz ders notları

7. LinkLabel



Uygulamadan bir dosyaya, klasöre ya da internet sayfasına yönlendirme yapmak için kullanılır. Tek bir LinkLabel üzerine birden fazla bağlantı verilebilir. Bağlantıya tıklanınca **LinkClicked** olayı tetiklenir.

Özellik / Üye	Açıklama
`Text`	Görünen yazı
`Links`	Bağlantılar koleksiyonu: `Links.Add(başlangıç, uzunluk, adres)`
`LinkColor`	Bağlantı rengi
`VisitedLinkColor`	Ziyaret edilmiş bağlantı rengi

```
private void Form1_Load(object sender, EventArgs e)
{
    string link = "https://www.firat.edu.tr";
    linkLabel1.Text = link;
    linkLabel1.Links.Add(0, link.Length, link);
}

private void linkLabel1_LinkClicked(object sender, LinkLabelLinkClickedEventArgs e)
{
    System.Diagnostics.Process.Start(e.Link.LinkData.ToString());
    linkLabel1.LinkVisited = true;      // bağlantı "ziyaret edildi" rengine geçer
}
```

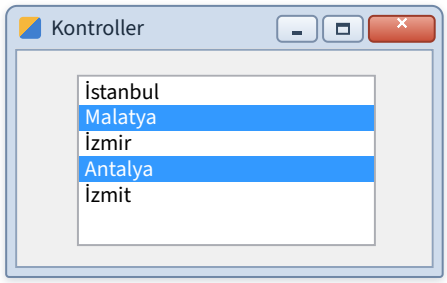
NOT

`Process.Start(adres)` yazımı .NET Framework projelerinde adresi varsayılan tarayıcıda açar. Yeni **.NET** (Core, 5 ve sonrası) şablonlarıyla açılan projelerde ise aynı iş şu satırla yapılır:

```
// .NET Core / .NET 5 ve sonrası projelerde adres şu şekilde açılır:
System.Diagnostics.Process.Start(new
System.Diagnostics.ProcessStartInfo("https://www.firat.edu.tr") { UseShellExecute = true
});
```


KampüsCepte · öğrenciler için ücretsiz ders notları

8. ListBox

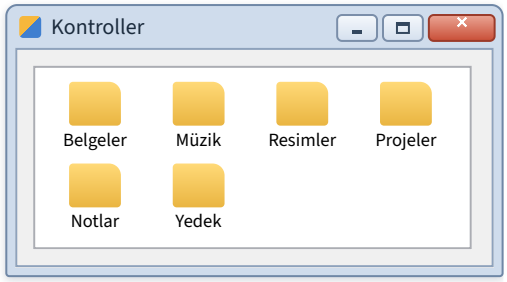


Kullanıcının bir veya daha fazla seçim yapabildiği ve bu seçimler üzerinde işlem yapılabilen bir liste sunar. Ekleme ve silme işlemleri 4. bölümde işlenecektir.

Özellik / Üye	Açıklama
`Items`	Elemanlar: `Add`, `AddRange`, `Clear`, `Remove`, `RemoveAt`
`SelectedItem`	/ `SelectedIndex` seçili eleman ve indisi
`SelectionMode`	One, MultiSimple, MultiExtended (çoklu seçim)
`Sorted`	`true` ise elemanlar alfabetik sıralanır

```
private void Form1_Load(object sender, EventArgs e)
{
    listBox1.Items.AddRange(new string[] { "İstanbul", "Malatya", "İzmir", "Antalya", "İzmit"
});
    listBox1.SelectionMode = SelectionMode.MultiExtended;   // Ctrl/Shift ile çoklu seçim
}
```

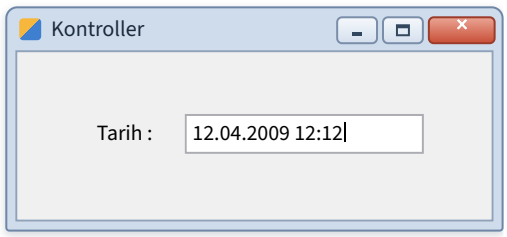
9. ListView



Farklı görünüm tipleri içeren bir liste yapısı sunar. Windows Gezgini'ndeki klasör görünümü buna örnektir: aynı öğeler büyük simge, küçük simge, liste ya da ayrıntılı tablo olarak gösterilebilir.

Özellik / Üye	Açıklama
`View`	LargeIcon, SmallIcon, List, Details, Tile
`Items`	Satırlar (her biri `ListViewItem`)
`Columns`	Details görünümündeki sütun başlıkları

10. MaskedTextBox



Normal TextBox'tan farklı olarak kullanıcıyı belirlenmiş bir kalıba uygun yazmaya zorlar. Böylece hatalı giriş baştan engellenir.

Özellik / Üye	Açıklama
`Mask`	Giriş kalıbı
`Text`	Girilen metin
`MaskCompleted`	Kalıbın zorunlu yerleri doldurulduysa `true`

KampüsCepte · öğrenciler için ücretsiz ders notları

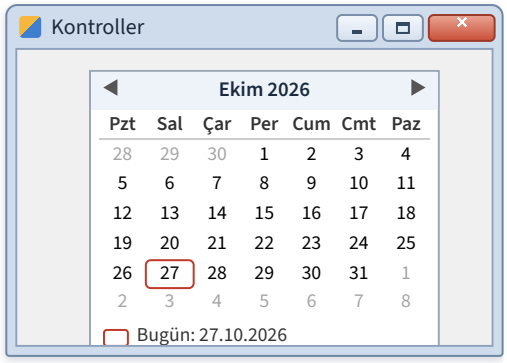
Mask karakteri	Anlamı
`0`	Rakam, zorunlu
`9`	Rakam ya da boşluk, isteğe bağlı
`L`	Harf, zorunlu
`?`	Harf, isteğe bağlı
`/` ve `:`	Tarih ve saat ayırıcısı (Türkçe sistemde tarih ayırıcısı nokta görünür)
`\`	Kendisinden sonraki karakteri kalıp karakteri değil, sabit yazı yapar

```
private void Form1_Load(object sender, EventArgs e)
{
    maskedTextBox1.Mask = "00/00/0000 90:00";   // tarih ve saat kalıbı
}

private void button1_Click(object sender, EventArgs e)
{
    if (maskedTextBox1.MaskCompleted)
        MessageBox.Show("Girilen: " + maskedTextBox1.Text);
    else
        MessageBox.Show("Lütfen kalıbı eksiksiz doldurunuz.");
}
```

11. MonthCalendar

DateTimePicker gibi formda takvim bulunmasını sağlar. Farkı, takvimi açılır kutu yerine doğrudan form üzerinde göstermesidir. Bir tarih aralığı da seçilebilir.



Özellik / Üye	Açıklama
`ShowWeekNumbers`	Hafta numaralarını gösterir
`ShowToday`	Alttaki *Bugün* yazısını gösterir
`ShowTodayCircle`	Bugünün tarihini çerçeve içine alır
`TodayDate`	Takvimin "bugün" kabul ettiği tarih

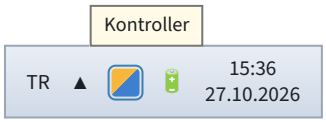
```
private void Form1_Load(object sender, EventArgs e)
{
    monthCalendar1.ShowWeekNumbers = true;
    monthCalendar1.ShowToday = true;
    monthCalendar1.ShowTodayCircle = true;
}

private void monthCalendar1_DateSelected(object sender, DateRangeEventArgs e)
{
    MessageBox.Show("Seçilen gün: " + e.Start.ToShortDateString());
}
```


KampüsCepte · öğrenciler için ücretsiz ders notları

12. NotifyIcon

Uygulamanın simgesini görev çubuğunun sağındaki bildirim alanında (saatin yanında) gösterir. Arka planda çalışan programlarda kullanılır.

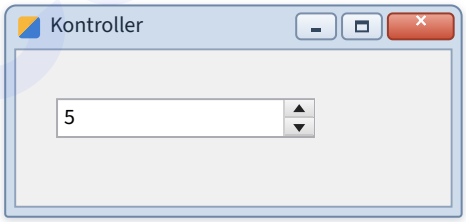


Özellik / Üye	Açıklama
`Icon`	Gösterilecek simge (atanmazsa görünmez)
`Text`	Fare üzerine gelince çıkan yazı
`Visible`	Simgenin görünmesi için `true` olmalıdır

```
private void Form1_Load(object sender, EventArgs e)
{
    notifyIcon1.Icon = this.Icon;        // simge atanmazsa bildirim alanında görünmez
    notifyIcon1.Text = "Kontroller";     // fare üzerine gelince görünen yazı
    notifyIcon1.Visible = true;
}
```

13. NumericUpDown

Form üzerinde, yukarı/aşağı oklarla değiştirilebilen bir sayı sayacı oluşturur. Kullanıcının sayı dışında bir şey yazmasını engeller.



Özellik / Üye	Açıklama
`Value`	Güncel değer (**decimal** türünde)
`Minimum`	/ `Maximum` alt ve üst sınır (varsayılan 0 ve 100)
`Increment`	Her ok tıklamasındaki artış miktarı
`DecimalPlaces`	Gösterilecek ondalık basamak sayısı

```
private void Form1_Load(object sender, EventArgs e)
{
    numericUpDown1.Minimum = 1;
    numericUpDown1.Maximum = 10;
    numericUpDown1.Increment = 1;
    numericUpDown1.Value = 5;
}

private void button1_Click(object sender, EventArgs e)
{
    int adet = Convert.ToInt32(numericUpDown1.Value);   // Value decimal türündedir
    MessageBox.Show(adet + " adet seçildi.");
}
```

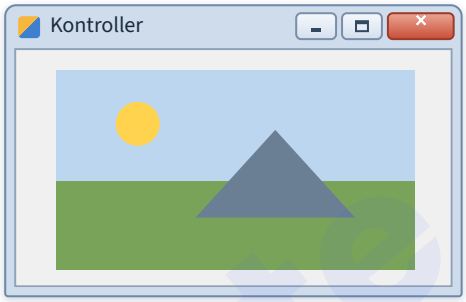

KampüsCepte · öğrenciler için ücretsiz ders notları

DİKKAT

`Value` değeri `Minimum` – `Maximum` aralığı dışında bir sayıya ayarlanırsa program çalışırken **ArgumentOutOfRangeException** hatası oluşur. Aynı kural ProgressBar için de geçerlidir.

14. PictureBox

Form üzerinde resim göstermek için kullanılır. Resim, tasarımda **Image** özelliğinin yanındaki düğmeyle bilgisayardan seçilir.

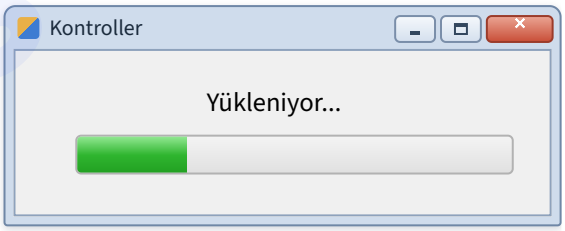


Özellik / Üye	Açıklama
`Image`	Gösterilecek resim
`SizeMode`	Normal, StretchImage, Zoom, CenterImage, AutoSize

```
private void Form1_Load(object sender, EventArgs e)
{
    pictureBox1.SizeMode = PictureBoxSizeMode.Zoom;     // resmi oranını bozmadan sığdırır
}
```

15. ProgressBar

Bir işlemin ilerleme durumunu adım adım gösterir (dosya kopyalama, yükleme gibi).



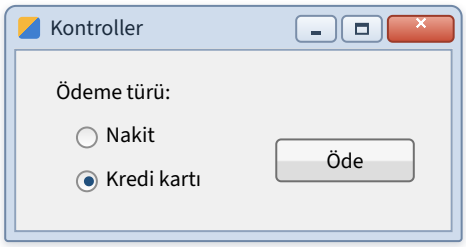
Özellik / Üye	Açıklama
`Value`	Güncel değer
`Minimum`	/ `Maximum` alt ve üst sınır
`Style`	Blocks, Continuous, Marquee (süresi belirsiz işlemler)

```
private void button1_Click(object sender, EventArgs e)
{
    if (progressBar1.Value + 25 <= progressBar1.Maximum)
    {
        progressBar1.Value += 25;          // her tıklamada %25 ilerler
    }
    else
    {
        MessageBox.Show("İşlem tamamlandı.");
    }
}
```


KampüsCepte · öğrenciler için ücretsiz ders notları

16. RadioButton

Bir grup seçenekten **yalnızca birinin** seçilmesini sağlar. Aynı kapsayıcıdaki (form ya da GroupBox) radyo butonlardan biri seçilince diğerlerinin seçimi kendiliğinden kalkar. Bu özelliğiyle CheckBox'tan ayrılır.

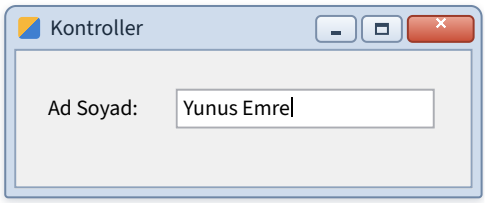


Özellik / Üye	Açıklama
`Text`	Seçeneğin yazısı
`Checked`	Seçili ise `true`
`CheckedChanged`	Seçim durumu değişince tetiklenen olay

```
private void button1_Click(object sender, EventArgs e)
{
    if (radioButton1.Checked)
        MessageBox.Show("Nakit ödeme seçildi.");
    else if (radioButton2.Checked)
        MessageBox.Show("Kredi kartı seçildi.");
    else
        MessageBox.Show("Lütfen bir ödeme türü seçiniz.");
}
```

17. TextBox

Kullanıcıdan metin almak için kullanılan temel kontroldür. Kutuya yazılan değer her zaman **string** türündedir. Sayı olarak kullanılacaksa dönüştürülmesi gerekir (3. bölüm). Özellikleri Bölüm 2.3'te verilmiştir.

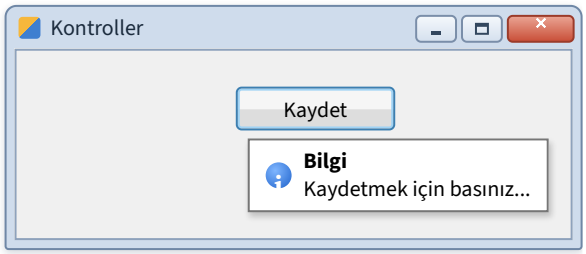


Özellik / Üye	Açıklama
`Text`	Kutudaki metin
`TextChanged`	Metin her değiştiğinde tetiklenen olay
`Clear()`	Kutuyu boşaltan metot
`Focus()`	İmleci kutuya getiren metot

KampüsCepte · öğrenciler için ücretsiz ders notları

18. ToolTip

Fare bir kontrolün üzerine geldiğinde açıklama gösterir. Forma eklendiğinde formun üzerinde değil, tasarım penceresinin altındaki bileşen tepsisinde görünür. Tek bir ToolTip, formdaki tüm kontroller için kullanılabilir.

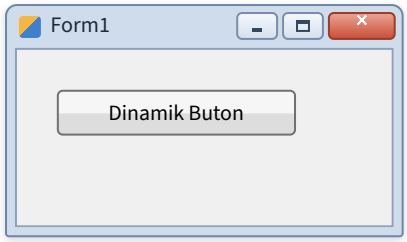


Özellik / Üye	Açıklama
`SetToolTip(kontrol, yazı)`	**Metottur.** Bir kontrole açıklama atar
`ToolTipTitle`	Açıklama kutusunun başlığı
`ToolTipIcon`	Başlık simgesi: Info, Warning, Error

```
private void Form1_Load(object sender, EventArgs e)
{
    toolTip1.SetToolTip(button1, "Kaydetmek için basınız...");
    toolTip1.ToolTipTitle = "Bilgi";
    toolTip1.ToolTipIcon = ToolTipIcon.Info;
}
```

2.5 Nesne Tabanlı Bakış: Kod ile Kontrol Oluşturma

Toolbox'tan sürüklenen her kontrol için Visual Studio, Designer dosyasına `this.button1 = new System.Windows.Forms.Button();` benzeri satırlar yazar. Aynı işi biz de kod ile, program çalışırken yapabiliriz. Bu, kontrollerin gerçekten birer nesne olduğunu en açık biçimde gösteren örnektir:




```
private void Form1_Load(object sender, EventArgs e)
{
    Button yeniButon = new Button();          // Button sınıfından yeni bir nesne
    yeniButon.Text = "Dinamik Buton";
    yeniButon.Location = new Point(20, 20);
    yeniButon.Width = 120;
    yeniButon.Click += YeniButon_Click;       // olaya metot bağlanır
    this.Controls.Add(yeniButon);             // nesne formun kontrollerine eklenir
}

private void YeniButon_Click(object sender, EventArgs e)
{
    MessageBox.Show("Kod ile oluşturulan butona tıklandı.");
}
```


KampüsCepte · öğrenciler için ücretsiz ders notları

1. `new Button()` ile **Button sınıfından** yeni bir nesne oluşturulur. Bu nesne şu an yalnızca bellektedir.
2. Nesnenin özellikleri (`Text`, `Location`, `Width`) ayarlanır.
3. `+=` işleci ile nesnenin **Click** olayına bir metot bağlanır. Designer dosyasındaki olay bağlantısı da tam olarak böyle yazılır.
4. `this.Controls.Add(...)` ile nesne formun kontrol koleksiyonuna eklenir ve ancak bu satırdan sonra ekranda görünür.

ÖZET

- Her kontrol bir **sınıftır**. Forma eklenen `button1`, `textBox1` gibi öğeler bu sınıfların **nesneleridir**.
- Özellikler tasarımda Properties penceresinden, çalışma anında kod ile değiştirilir. Properties ayarları Designer dosyasına kod olarak yazılır.
- `Visible = false` kontrolü gizler. `Enabled = false` ise kontrolü görünür bırakır ama pasif yapar. Özelliğin adı **Enabled**'dır.
- Tek seçim için **RadioButton** ya da **ComboBox**, çoklu seçim için **CheckBox** ya da **CheckedListBox** kullanılır. Kalıplı giriş için **MaskedTextBox**, sınırlı sayı girişi için **NumericUpDown**, tarih için **DateTimePicker** seçilir.
- `GetItemChecked`, `SetToolTip`, `Clear`, `Focus` birer **metottur** ve parantezle çağrılır. Bunlara `=` ile değer atanamaz.
- NumericUpDown'ın `Value` özelliği **decimal** türündedir. Value ile ProgressBar değerleri sınırların dışına çıkarsa çalışma anında hata oluşur.
- Kontroller kod ile de oluşturulabilir: `new` → özellikleri ayarla → olaya bağla (`+=`) → `Controls.Add`.

KampüsCepte

KampüsCepte · öğrenciler için ücretsiz ders notları